Figure 16.9 With client-side validation, clicking Submit triggers validation to be shown in the browser before the request is sent to the server. As shown in the right pane, no request is sent.

If you're building an SPA, the onus is on the client-side framework to validate the data on the client side before posting it to the API. The API must still validate the data when it arrives at the server, but the client-side framework is responsible for providing the smooth user experience.

When you use Razor Pages to generate your HTML, you get much of this validation code for free. Razor Pages automatically configures client-side validation for most of the built-in attributes without requiring additional work, as you'll see in chapter 18.

Unfortunately, if you've used custom `ValidationAttributes`, these will run only on the server by default; you need to do some additional wiring up of the attribute to make it work on the client side too. Despite this, custom validation attributes can be useful for handling common validation scenarios in your application, as you'll see in chapter 31.

The model binding framework in ASP.NET Core gives you a lot of options on how to organize your Razor Pages: page handler parameters or `PageModel` properties; one binding model or multiple; options for where to define your binding model classes. In the next section I give some advice on how *I* like to organize my Razor Pages.

16.4 Organizing your binding models in Razor Pages

In this section I give some general advice on how I like to configure the binding models in my Razor Pages. If you follow the patterns in this section,